

CSA09 – PROGRAMMING IN JAVA

COMMON ASSIGNMENT REPORT

Smart Library Management, Book Reservation and Overdue Notification System using Java

Department : Computer Science and Engineering

Team Members and Register Numbers:

R. Kaviya(192524154)

M. Swarna Ambica(192472390)

G. Hasini(192425262)

Yuvashree(192521485)

Faculty name : Dr. R. Thalpathi Rajasekaran

Academic year: 2024&2025

1. Problem Statement and Problem Formulation

The Smart Library Management, Book Reservation and Overdue Notification System is a menu-driven Java application designed to automate important library activities. The system manages members, books, inventory, book issue and return operations, reservations, overdue notifications, fines and reports. Since book stock is a limited resource, the system must maintain accurate available-copy information even when concurrent operations are simulated.

The problem is formulated as follows: design and implement a Java application in which members can register, search and borrow books, reserve books that are unavailable, return books and receive due-date or overdue notifications. Librarians can manage member and book records and generate circulation, fine and inventory-utilization reports. The implementation must demonstrate the Java concepts specified by the course outcomes, including object-oriented programming, collections, exception handling, multithreading, GUI/event handling, file I/O, serialization, hashing and JDBC.

2. Objectives

- To develop a reliable Java-based library management application.
- To apply classes, objects, encapsulation, data hiding, inheritance and polymorphism.
- To use Java Collection Framework classes such as ArrayList, Set, HashMap and Hashtable with generics.
- To implement Iterator/ListIterator for traversal, searching, updating and removal.
- To handle invalid input, invalid member IDs, unavailable books and duplicate reservations using built-in and user-defined exceptions.
- To simulate concurrent book-issue and overdue-notification activities using threads, priorities, synchronization and inter-thread communication.
- To provide a clean menu-driven GUI using Java AWT/Swing and event handling.
- To use Java I/O, serialization and hashing concepts and connect the application to a database using JDBC.
- To generate meaningful circulation, fine and inventory-utilization reports.

3. Expected Outcomes

- A working library management application with member and book management.
- Accurate tracking of book inventory and available copies.
- A reservation waitlist for unavailable books.
- Automatic due-date and overdue notification logic.
- Fine calculation based on overdue days.
- Robust exception handling without unnecessary program termination.
- Demonstration of all mapped Java course concepts.
- Database-backed storage and retrieval using JDBC.

4. Requirements, Constraints and Assumptions

4.1 Functional Requirements

- Register, search, update and delete member records.
- Add, search, update and delete book records.
- Display total and available book copies.
- Issue and return books.
- Reserve unavailable books and maintain a waiting list.
- Cancel reservations.
- Check due dates and identify overdue books.
- Generate notifications and calculate fines.
- Generate circulation, fine and inventory reports.
- Store and retrieve data through JDBC.

4.2 Non-Functional Requirements

- The interface should be simple and understandable.
- The system should validate inputs before database operations.
- Shared inventory data must remain consistent during concurrent tasks.
- The code should be modular and maintainable.
- The application should handle common exceptional conditions gracefully.

4.3 Constraints and Assumptions

- The system is developed for academic demonstration.
- A relational database such as MySQL is assumed for JDBC operations.
- A fixed fine-per-day value may be configured for the prototype.
- Users are assumed to provide valid information unless validation rejects the input.
- Threading is used to simulate concurrency rather than to represent a production-scale distributed system.

5. Course Concepts Applied

CO	Concept	Project Application
CO1	OOP	Member, Book, Reservation, Inventory and Notification classes; encapsulation, inheritance and polymorphism.
CO2	Collections	ArrayList, Set/HashSet, HashMap, Hashtable, generics, Iterator and ListIterator.
CO3	Threads & Exceptions	Issue and notification threads, priorities, synchronization, wait/notify and custom exceptions.
CO4	GUI & Events	AWT/Swing components, layouts, menus and

		ActionListener-based event handling.
CO5	I/O, Serialization, Hashing & JDBC	Text reports, object serialization, hashed credentials/keys where appropriate, and database CRUD operations.

6. System Design / Proposed Solution

The proposed system follows a modular design. The model layer contains core entities, the collection layer manages in-memory records, the service layer performs library operations, the GUI layer provides user interaction, the threading layer handles concurrent tasks, and the database layer performs JDBC operations.

6.1 Main Modules

- Member Management – registration, search, update and deletion.
- Book Management – adding, searching, updating and deleting books.
- Inventory Management – tracking total and available copies.
- Issue/Return Management – processing circulation and updating stock.
- Reservation Management – maintaining a waitlist for unavailable books.
- Overdue Notification – checking due dates and generating reminders/overdue messages.
- Fine Management – calculating fines from overdue days.
- Reporting – circulation, fine and inventory-utilization reports.
- Database Management – JDBC-based INSERT, SELECT, UPDATE and DELETE operations.

6.2 Core Classes

Class	Purpose
Member	Stores member identity and membership information.
RegularMember	Defines regular-member borrowing behaviour.
PremiumMember	Defines premium-member borrowing behaviour.
Book	Stores book details and category information.
Reservation	Stores reservation and waitlist information.
Inventory	Maintains book stock and available copies.
Notification	Represents due-date and overdue messages.
IssueRecord	Stores circulation details such as issue and due dates.
Library	Coordinates major library operations.
DatabaseConnection	Creates and manages the JDBC connection.

7. Object-Oriented Design

7.1 Encapsulation and Data Hiding

Core entity fields are declared private and accessed through constructors, getters and setters. This prevents direct uncontrolled modification of important data such as member IDs and available copies.

Example:

```
class Book { private int bookId; private String title; private int availableCopies; }
```

7.2 Inheritance and Polymorphism

Different membership tiers can inherit from a common Member class. The borrowing-limit method can be overridden so that the actual behaviour depends on the runtime object.

Example:

```
Member m1 = new RegularMember();  
Member m2 = new PremiumMember();  
m1.getBorrowLimit();  
m2.getBorrowLimit();
```

8. Collection Framework Design

Collection	Use in System
ArrayList<Book>	Stores and traverses book objects.
HashSet<Integer>	Maintains unique member IDs or reservation identifiers.
HashMap<Integer, Member>	Maps member IDs to member objects for fast lookup.
Hashtable<Integer, Integer>	Stores synchronized-style inventory mappings for demonstration.
Iterator<Book>	Traverses book records.
ListIterator<Book>	Supports bidirectional traversal and list updates.
Generics	Provides type safety for all collections.

9. Exception Handling

The system uses both built-in and user-defined exceptions.

Exception	Condition
InvalidMemberException	Member ID does not exist.
BookUnavailableException	No copy is available for issue.
DuplicateReservationException	Member attempts to reserve the same book again.
InvalidBookIdException	Book ID is invalid.
InvalidInputException	Input does not satisfy required validation.

try-catch-finally and appropriate validation are used so that invalid operations display useful messages while allowing the application to continue.

10. Multithreading Design

Two concurrent tasks are proposed: an Issue Processing Thread and an Overdue Notification Thread. Both may access shared inventory or reservation information. Synchronization is therefore used around critical sections that modify shared state.

Thread	Responsibility	Concurrency Feature
IssueThread	Processes book issue and inventory updates.	Synchronization and thread priority.
NotificationThread	Checks due dates and produces notifications.	Thread execution and inter-thread coordination.

The design can demonstrate Thread.MIN_PRIORITY, Thread.NORM_PRIORITY or Thread.MAX_PRIORITY, along with synchronized methods/blocks and wait()/notify() for coordination.

11. GUI and Event Handling

The application can use Java Swing/AWT to provide a clean menu-driven interface. Typical components include JFrame, JPanel, JLabel, JTextField, JButton, JTable, JComboBox, JOptionPane, menus and layout managers. ActionListener is used to connect user actions to operations.

Suggested main menu:

1. Register Member
2. Add Book
3. Search Book
4. Display Books
5. Issue Book
6. Return Book
7. Reserve Book
8. Cancel Reservation
9. View Overdue Books
10. Notifications
11. Fine Report
12. Inventory Report
13. Exit

12. Java I/O, Serialization and Hashing

- Character stream/FileWriter can be used to create text-based reports.
- ObjectOutputStream/ObjectInputStream can be used to serialize selected library objects for persistence demonstrations.
- HashMap/Hashtable demonstrate hashing-based key-value storage. A suitable hash-based representation can also be used where credentials or identifiers require hashing, without storing sensitive values in plain text.

13. JDBC Database Design

Table	Important Fields
MEMBER	member_id, name, email, membership_type
BOOK	book_id, title, author, category, total_copies, available_copies
ISSUE	issue_id, member_id, book_id, issue_date, due_date, return_date, status
RESERVATION	reservation_id, member_id, book_id, reservation_date, status
NOTIFICATION	notification_id, member_id, message, notification_date, status

JDBC is used for INSERT, SELECT, UPDATE and DELETE operations. PreparedStatement is recommended for parameterized database operations.

14. Overall Algorithm / Pseudocode

START

Initialize the library system and connect to the database.

Display the main menu.

WHILE the user has not selected Exit:

 Read the selected operation.

 IF member management is selected, perform add/search/update/delete.

 ELSE IF book management is selected, perform add/search/update/delete.

 ELSE IF issue is selected, validate member and book, check availability, update inventory and save issue record.

 ELSE IF return is selected, update return information, restore inventory and calculate overdue fine if applicable.

 ELSE IF reservation is selected, validate member, check availability and duplicate reservation, then add to waitlist if needed.

 ELSE IF overdue notification is selected, compare due dates and generate reminders/overdue messages.

 ELSE IF reports is selected, generate circulation, fine and inventory reports.

 ELSE display an invalid-choice message.

END WHILE

Close database connection.

END

15. Reservation Algorithm

START

Input Member ID and Book ID.

Check whether the member exists.

IF member does not exist, raise InvalidMemberException.

Check available copies.

IF copies are available, inform the user that the book can be issued.

ELSE check whether the member already has a reservation.
 IF duplicate, raise DuplicateReservationException.
 ELSE add the member to the reservation waitlist.
 END.

16. Overdue Notification Algorithm

START

Retrieve active issue records.

FOR each issue record:

Compare the current date with the due date.

IF current date is after due date:

Calculate overdue days.

Calculate fine = overdue days × fine-per-day.

Generate an overdue notification.

ELSE IF only a small number of days remain:

Generate a due-date reminder.

END FOR

STOP.

17. Environment and Tools

Component	Proposed Tool
Language	Java
JDK	Java 21 or compatible course-supported JDK
GUI	Java Swing / AWT
Database	MySQL
Connectivity	JDBC
IDE	VS Code / IntelliJ IDEA / Eclipse / NetBeans
Version Control	Git and GitHub
Documentation	MS Word / Google Docs

18. Implementation Structure

A modular project structure is recommended so that each course concept can be demonstrated clearly.

SmartLibrary/

src/model/Member.java, Book.java, Reservation.java, Inventory.java, Notification.java

src/dao/MemberDAO.java, BookDAO.java, ReservationDAO.java

src/gui/LoginFrame.java, Dashboard.java, BookFrame.java

src/threads/IssueThread.java, NotificationThread.java

src/exceptions/InvalidMemberException.java, BookUnavailableException.java,

DuplicateReservationException.java

src/database/DatabaseConnection.java

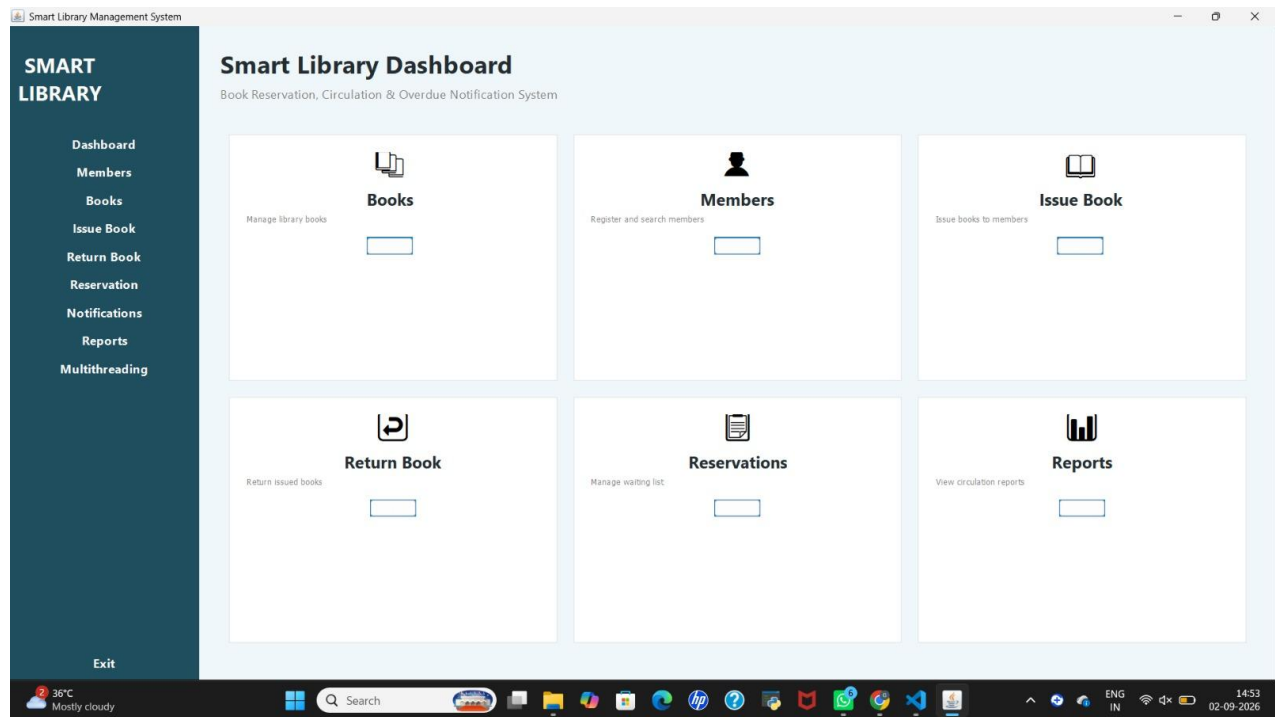
database/library.sql

screenshots/
README.md

19. Test Cases and Expected / Actual Results

TC	Test Input / Operation	Expected Result	Actual Result
TC01	Register valid member	Member registered	To be filled after execution
TC02	Register duplicate member ID	Duplicate/validation error	To be filled
TC03	Add valid book	Book added	To be filled
TC04	Search existing book	Book details displayed	To be filled
TC05	Issue available book	Issue successful; stock decreases	To be filled
TC06	Issue unavailable book	BookUnavailableException/message	To be filled
TC07	Reserve unavailable book	Reservation added to waitlist	To be filled
TC08	Duplicate reservation	DuplicateReservationException/message	To be filled
TC09	Return before due date	No overdue fine	To be filled
TC10	Return after due date	Fine and overdue notification	To be filled
TC11	Invalid member ID	InvalidMemberException/message	To be filled
TC12	Invalid menu option	Validation message	To be filled
TC13	Update book/member	Record updated	To be filled
TC14	Delete record	Record deleted	To be filled
TC15	Generate reports	Circulation/inventory/fine report	To be filled

20. Execution Screenshots / Outputs



21. Results and Validation

After implementation, validate the system using the test cases. Record the actual result for each test case and calculate the overall success rate as: $\text{Test Success Rate} = (\text{Number of Passed Tests} / \text{Total Tests}) \times 100$. Inventory validation should also confirm that available copies never become negative and that issuing and returning a book update stock consistently.

Metric	Value to Record
Total test cases	15
Passed test cases	Fill after execution
Failed test cases	Fill after execution
Test success rate	Calculate after execution
Total books	Fill from application
Issued books	Fill from application
Reserved books	Fill from application
Overdue books	Fill from application

22. Analysis, Comparison, Trade-offs and Justification

Design Choice	Reason	Trade-off
ArrayList	Simple ordered storage and traversal.	Lookup by key is slower than HashMap.
HashMap	Fast key-based member/book lookup.	Does not provide list-style ordering.
HashSet	Prevents duplicate reservation/member identifiers.	Does not preserve list ordering.
Hashtable	Demonstrates the required legacy synchronized map concept.	Generally less flexible than modern concurrent collections.
JDBC + MySQL	Persistent and structured storage.	Requires database configuration and connection handling.
Swing/AWT	Provides a visible desktop interface and event handling.	More code than a console-only application.

Synchronization	Protects shared inventory from race conditions.	Can reduce concurrency when critical sections are large.
-----------------	---	--

The final solution balances the academic requirements with implementation simplicity. The system intentionally demonstrates the course concepts inside realistic library workflows.

23. SDG Relevance

SDG	Relevance to Project
SDG 4 – Quality Education	Supports organized access to educational resources and efficient library services.

SDG 9 – Industry, Innovation and Infrastructure	Demonstrates digital automation, software engineering and information infrastructure.
SDG 16 – Peace, Justice and Strong Institutions	Promotes accountable record management, controlled access and reliable institutional processes.

24. Conclusion

The proposed Smart Library Management, Book Reservation and Overdue Notification System provides an integrated Java solution for managing library members, books, inventory, circulation, reservations and overdue notifications. The design directly addresses the assignment requirements and provides a practical context for demonstrating Java OOP, collections, exception handling, multithreading, GUI/event handling, I/O, serialization, hashing and JDBC.

25. Limitations

- The prototype is intended for academic use and may not include enterprise-level authentication.
- Notification delivery may be simulated through application messages rather than real email/SMS.
- The threading model is a controlled simulation of concurrent tasks.
- Fine rules may be simplified for demonstration.

26. Possible Improvements / Future Scope

- Add role-based authentication for librarian and member accounts.
- Add email/SMS notification integration.
- Add advanced search and filtering.
- Add graphical analytics for circulation and inventory utilization.
- Use a modern database connection pool for larger deployments.
- Add automated unit and integration testing.
- Add cloud deployment and a web/mobile client in a future version.

27. Individual Contribution of Group Members

Member	Contribution
Student 1: _____	OOP class design, database/JDBC and core library operations.
Student 2: _____	GUI/event handling, collections and reservation module.
Student 3: _____	Multithreading, exception handling, testing and reports.

Replace the above with the team's actual contribution. Each member must have a genuine and clearly stated contribution.

28. Individual Reflection – Student 1

During this assignment, I contributed to the design and development of the Smart Library Management System. One of the main design decisions was to divide the application into separate classes so that each entity had a clear responsibility. I learned how encapsulation protects data and how inheritance and method overriding can support different membership behaviours. I also gained practical experience with collections such as ArrayList, HashMap and Set for storing and retrieving library records.

A major challenge was maintaining consistency when more than one operation could access shared inventory data. This helped me understand why synchronization is important in multithreaded programs. Handling invalid member IDs, unavailable books and duplicate reservations also showed me how user-defined exceptions improve robustness. The project connected strongly with SDG 4 because a well-organized library system improves access to educational resources. It also relates to SDG 9 through software automation and SDG 16 through reliable institutional record management.

Overall, the assignment improved my understanding of Java programming beyond individual examples. I learned to combine OOP, collections, exceptions, threads, GUI concepts and JDBC into one application. The experience also improved my ability to test, debug, document and justify design decisions.

29. Individual Reflection – Student 2

I contributed to the implementation and testing of the library system, with particular attention to the user interface, collections and reservation workflow. Designing a menu-driven interface helped me understand event-driven programming and the role of listeners in Java GUI applications. I also learned how collections can be selected according to the operation required: ArrayList for ordered records, HashMap for key-based lookup and Set for uniqueness.

The reservation workflow presented challenges because the system must prevent duplicate reservations while still maintaining a useful waiting list. Exception handling provided a structured way to deal with these cases. Testing different inputs also helped identify conditions that were not obvious during initial implementation. The project supports SDG 4 by improving library resource management and SDG 9 by applying programming and digital infrastructure to an institutional problem.

The assignment strengthened my ability to connect Java theory with a complete application. I learned that good software design is not only about making the program run; it also requires clear responsibilities, validation, error handling, testing and documentation.

30. Individual Reflection – Student 3

My contribution focused on multithreading, exception handling, testing and report generation. The assignment helped me understand how concurrent tasks can be simulated using Java threads and why shared inventory information needs synchronization. Thread priorities and inter-thread communication also gave me practical exposure to concepts that are otherwise difficult to understand only from theory.

Another important learning area was exception handling. User-defined exceptions made it possible to represent library-specific errors such as unavailable books and duplicate reservations. I also participated in test-case preparation and validation, which demonstrated the importance of checking both normal and exceptional workflows. The project relates to SDG 4, SDG 9 and SDG 16 through educational resource access, digital innovation and reliable institutional processes.

Overall, the project improved my ability to analyse a problem, divide it into modules, implement Java features in a meaningful context and evaluate the resulting system. It also highlighted the importance of teamwork and clear individual responsibility.

31. References

- Course assignment brief: CSA09 – Programming in Java (SLOT A), provided by the course/institution.
- Java Platform Standard Edition API documentation – for Java language, collections, threads, I/O and GUI APIs.
- MySQL documentation – for relational database and SQL concepts used with JDBC.
- JDBC API documentation – for database connectivity and CRUD operations.
- GitHub documentation – for repository management and source-code version control.

32. Appendices

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.concurrent.*;

public class SmartLibrary extends JFrame {

    // CHANGE THESE IF YOUR MYSQL LOGIN IS DIFFERENT
    static final String USER = "root";
    static final String PASS = "root";

    static final String DB = "smart_library";

    static final String SERVER =
        "jdbc:mysql://localhost:3306/?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";

    static final String URL =
        "jdbc:mysql://localhost:3306/" + DB +
        "?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";

    JPanel main;
```

```
ExecutorService pool = Executors.newFixedThreadPool(3);
```

```
// ===== JDBC =====
```

```
static Connection con() throws Exception {  
    Class.forName("com.mysql.cj.jdbc.Driver");  
    return DriverManager.getConnection(URL, USER, PASS);  
}
```

```
static void initDB() throws Exception {
```

```
    Class.forName("com.mysql.cj.jdbc.Driver");
```

```
    try (Connection c = DriverManager.getConnection(SERVER, USER, PASS);  
        Statement s = c.createStatement()) {
```

```
        s.executeUpdate("CREATE DATABASE IF NOT EXISTS " + DB);  
    }
```

```
    try (Connection c = con();  
        Statement s = c.createStatement()) {
```

```
        s.executeUpdate(  
            "CREATE TABLE IF NOT EXISTS members (" +  
            "member_id INT PRIMARY KEY," +  
            "name VARCHAR(100) NOT NULL," +  
            "email VARCHAR(150)," +  
            "membership_type VARCHAR(30))"  
        );
```

```
        s.executeUpdate(  
            "CREATE TABLE IF NOT EXISTS books (" +  
            "book_id INT PRIMARY KEY," +  
            "title VARCHAR(200) NOT NULL," +  
            "author VARCHAR(150)," +  
            "category VARCHAR(80)," +  
            "total_copies INT NOT NULL," +  
            "available_copies INT NOT NULL)"  
        );
```

```

s.executeUpdate(
    "CREATE TABLE IF NOT EXISTS issues (" +
    "issue_id INT AUTO_INCREMENT PRIMARY KEY," +
    "member_id INT NOT NULL," +
    "book_id INT NOT NULL," +
    "issue_date DATE," +
    "due_date DATE," +
    "return_date DATE," +
    "fine DECIMAL(10,2) DEFAULT 0," +
    "FOREIGN KEY(member_id) REFERENCES members(member_id)," +
    "FOREIGN KEY(book_id) REFERENCES books(book_id))"
);

```

```

s.executeUpdate(
    "CREATE TABLE IF NOT EXISTS reservations (" +
    "reservation_id INT AUTO_INCREMENT PRIMARY KEY," +
    "member_id INT NOT NULL," +
    "book_id INT NOT NULL," +
    "reservation_date DATE," +
    "status VARCHAR(30) DEFAULT 'WAITLIST'," +
    "FOREIGN KEY(member_id) REFERENCES members(member_id)," +
    "FOREIGN KEY(book_id) REFERENCES books(book_id))"
);

```

```

}

```

```

}

```

```

// ===== WINDOW =====

```

```

public SmartLibrary() {

```

```

    setTitle("Smart Library Management System");
    setSize(1150, 700);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

```

```

    add(sidebar(), BorderLayout.WEST);

```

```

    main = new JPanel(new BorderLayout());
    add(main, BorderLayout.CENTER);

```

```
    dashboard();  
}
```

```
JPanel sidebar() {
```

```
    JPanel p = new JPanel();  
    p.setPreferredSize(new Dimension(190, 0));  
    p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));  
    p.setBackground(new Color(20, 65, 82));
```

```
    JLabel title = new JLabel(  
        "<html><center>SMART<br>LIBRARY</center></html>"  
    );
```

```
    title.setForeground(Color.WHITE);  
    title.setFont(new Font("SansSerif", Font.BOLD, 22));  
    title.setAlignmentX(Component.CENTER_ALIGNMENT);
```

```
    p.add(Box.createVerticalStrut(20));  
    p.add(title);  
    p.add(Box.createVerticalStrut(20));
```

```
    String[] names = {  
        "Dashboard",  
        "Members",  
        "Books",  
        "Search Books",  
        "Issue Book",  
        "Return Book",  
        "Reservation",  
        "Notifications",  
        "Reports",  
        "JDBC Output",  
        "Exception Test",  
        "Multithreading"  
    };
```

```
    Runnable[] actions = {  
        this::dashboard,
```

```
this::members,  
this::books,  
this::searchBooks,  
this::issueBook,  
this::returnBook,  
this::reservation,  
this::notifications,  
this::reports,  
this::jdbcOutput,  
this::exceptionTest,  
this::multithreading  
};
```

```
for (int i = 0; i < names.length; i++) {
```

```
    JButton b = new JButton(names[i]);
```

```
    b.setMaximumSize(new Dimension(185, 38));  
    b.setAlignmentX(Component.CENTER_ALIGNMENT);  
    b.setForeground(Color.WHITE);  
    b.setBackground(new Color(20, 65, 82));  
    b.setBorderPainted(false);  
    b.setFocusPainted(false);
```

```
    Runnable action = actions[i];  
    b.addActionListener(e -> action.run());
```

```
    p.add(b);  
}
```

```
p.add(Box.createVerticalGlue());
```

```
JButton exit = new JButton("Exit");
```

```
exit.setAlignmentX(Component.CENTER_ALIGNMENT);  
exit.addActionListener(e -> System.exit(0));
```

```
p.add(exit);  
p.add(Box.createVerticalStrut(10));
```

```

    return p;
}

void page(String heading, String sub) {

    main.removeAll();

    JPanel top = new JPanel();
    top.setLayout(new BoxLayout(top, BoxLayout.Y_AXIS));

    top.setBorder(
        BorderFactory.createEmptyBorder(20, 25, 10, 20)
    );

    JLabel h = new JLabel(heading);
    h.setFont(new Font("SansSerif", Font.BOLD, 28));

    top.add(h);
    top.add(Box.createVerticalStrut(5));
    top.add(new JLabel(sub));

    main.add(top, BorderLayout.NORTH);
}

void refresh() {
    main.revalidate();
    main.repaint();
}

JButton button(String s) {
    return new JButton(s);
}

JPanel form() {

    JPanel p = new JPanel(new GridBagLayout());

    p.setBorder(
        BorderFactory.createEmptyBorder(10, 30, 20, 30)
    );

```

```

        return p;
    }

    void row(
        JPanel p,
        GridBagConstraints g,
        int y,
        String label,
        JComponent c
    ) {

        g.gridx = 0;
        g.gridy = y;
        g.weightx = 0;
        g.fill = GridBagConstraints.NONE;

        p.add(new JLabel(label), g);

        g.gridx = 1;
        g.weightx = 1;
        g.fill = GridBagConstraints.HORIZONTAL;

        p.add(c, g);
    }

    // ===== DASHBOARD =====

    void dashboard() {

        page(
            "Smart Library Dashboard",
            "Book Reservation, Circulation & Overdue Notification System"
        );

        JPanel grid = new JPanel(
            new GridLayout(2, 3, 18, 18)
        );

        grid.setBorder(

```

```

    BorderFactory.createEmptyBorder(20, 25, 25, 25)
);

String[][] cards = {
    {"BOOKS", "Books", "Add, search, update and delete books"},
    {"MEMBERS", "Members", "Register and manage members"},
    {"ISSUE", "Issue Book", "Issue books to members"},
    {"RETURN", "Return Book", "Return books and calculate fine"},
    {"RESERVE", "Reservations", "Waiting list and cancellation"},
    {"REPORT", "Reports", "Inventory and circulation reports"}
};

for (String[] c : cards) {

    JPanel card = new JPanel(new GridLayout(3, 1));

    card.setBorder(
        BorderFactory.createLineBorder(Color.LIGHT_GRAY)
    );

    JLabel icon =
        new JLabel(c[0], SwingConstants.CENTER);

    icon.setFont(
        new Font("SansSerif", Font.BOLD, 20)
    );

    JLabel name =
        new JLabel(c[1], SwingConstants.CENTER);

    name.setFont(
        new Font("SansSerif", Font.BOLD, 18)
    );

    JLabel desc =
        new JLabel(c[2], SwingConstants.CENTER);

    desc.setFont(
        new Font("SansSerif", Font.PLAIN, 11)
    );
}

```

```

        card.add(icon);
        card.add(name);
        card.add(desc);

        grid.add(card);
    }

    main.add(grid, BorderLayout.CENTER);

    refresh();
}

// ===== MEMBERS =====

void members() {

    page(
        "Member Management",
        "Add, search, update and delete members using JDBC"
    );

    JPanel p = form();

    JTextField id = new JTextField();
    JTextField name = new JTextField();
    JTextField email = new JTextField();

    JComboBox<String> type =
        new JComboBox<>(
            new String[] {"Student", "Faculty", "Staff"}
        );

    GridBagConstraints g =
        new GridBagConstraints();

    g.insets = new Insets(7, 7, 7, 7);

    row(p, g, 0, "Member ID", id);
    row(p, g, 1, "Name", name);

```

```
row(p, g, 2, "Email", email);
row(p, g, 3, "Membership Type", type);
```

```
JPanel buttons = new JPanel();
```

```
JButton add = button("Add");
JButton update = button("Update");
JButton del = button("Delete");
JButton find = button("Search");
```

```
buttons.add(add);
buttons.add(update);
buttons.add(del);
buttons.add(find);
```

```
g.gridx = 1;
g.gridy = 4;
```

```
p.add(buttons, g);
```

```
JTextArea out = new JTextArea(6, 55);
```

```
out.setEditable(false);
```

```
g.gridy = 5;
```

```
p.add(new JScrollPane(out), g);
```

```
main.add(p);
```

```
// ADD MEMBER
```

```
add.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "INSERT INTO members VALUES(?,?,?,?)"
            )
    )
```

```

    ) {

        s.setInt(
            1,
            Integer.parseInt(id.getText())
        );

        s.setString(2, name.getText());
        s.setString(3, email.getText());

        s.setString(
            4,
            (String) type.getSelectedItem()
        );

        s.executeUpdate();

        out.setText(
            "SUCCESS\nMember added successfully."
        );

    } catch (Exception ex) {

        out.setText(
            "ERROR: " + ex.getMessage()
        );
    }
});

// UPDATE MEMBER

update.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "UPDATE members SET name=?,email=?,membership_type=? WHERE member_id=?"
            )
    ) {

```

```

s.setString(1, name.getText());
s.setString(2, email.getText());

s.setString(
    3,
    (String) type.getSelectedItem()
);

s.setInt(
    4,
    Integer.parseInt(id.getText())
);

int n = s.executeUpdate();

out.setText(
    n > 0
    ? "SUCCESS\nMember updated."
    : "Member not found."
);

} catch (Exception ex) {

    out.setText(
        "ERROR: " + ex.getMessage()
    );
}
});

// DELETE MEMBER

del.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "DELETE FROM members WHERE member_id=?"
            )
    )

```

```

    ) {

        s.setInt(
            1,
            Integer.parseInt(id.getText())
        );

        int n = s.executeUpdate();

        out.setText(
            n > 0
            ? "SUCCESS\nMember deleted."
            : "Member not found."
        );

    } catch (Exception ex) {

        out.setText(
            "ERROR: " + ex.getMessage()
        );
    }
});

// SEARCH MEMBER

find.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "SELECT * FROM members WHERE member_id=?"
            )
    ) {

        s.setInt(
            1,
            Integer.parseInt(id.getText())
        );
    }
}

```

```
ResultSet r = s.executeQuery();
```

```
if (r.next()) {
```

```
    name.setText(  
        r.getString("name")  
    );
```

```
    email.setText(  
        r.getString("email")  
    );
```

```
    type.setSelectedItem(  
        r.getString("membership_type")  
    );
```

```
    out.setText("Member found.");
```

```
} else {
```

```
    out.setText("Member not found.");  
}
```

```
} catch (Exception ex) {
```

```
    out.setText(  
        "ERROR: " + ex.getMessage()  
    );  
}  
});
```

```
refresh();  
}
```

```
// ===== BOOKS =====
```

```
void books() {
```

```
    page(  
        "Book Management",  
        "Add, search, update and delete books"  
    );
```

```
JPanel p = form();

JTextField id = new JTextField();
JTextField title = new JTextField();
JTextField author = new JTextField();
JTextField category = new JTextField();
JTextField copies = new JTextField();

GridBagConstraints g =
    new GridBagConstraints();

g.insets = new Insets(7, 7, 7, 7);

row(p, g, 0, "Book ID", id);
row(p, g, 1, "Title", title);
row(p, g, 2, "Author", author);
row(p, g, 3, "Category", category);
row(p, g, 4, "Total Copies", copies);

JPanel buttons = new JPanel();

JButton add = button("Add");
JButton update = button("Update");
JButton del = button("Delete");
JButton find = button("Search");

buttons.add(add);
buttons.add(update);
buttons.add(del);
buttons.add(find);

g.gridx = 1;
g.gridy = 5;

p.add(buttons, g);

JTextArea out = new JTextArea(6, 55);

out.setEditable(false);

g.gridy = 6;

p.add(new JScrollPane(out), g);

main.add(p);
```

```
// ADD BOOK
```

```
add.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "INSERT INTO books VALUES(?,?,?,?,?,?)"
            )
    ) {

        int n =
            Integer.parseInt(copies.getText());

        s.setInt(
            1,
            Integer.parseInt(id.getText())
        );

        s.setString(2, title.getText());
        s.setString(3, author.getText());
        s.setString(4, category.getText());

        s.setInt(5, n);
        s.setInt(6, n);

        s.executeUpdate();

        out.setText(
            "SUCCESS\nBook added successfully."
        );

    } catch (Exception ex) {

        out.setText(
            "ERROR: " + ex.getMessage()
        );
    }
});
```

```
// UPDATE BOOK
```

```
update.addActionListener(e -> {
```

```

try (
    Connection c = con();
    PreparedStatement s =
        c.prepareStatement(
            "UPDATE books SET title=?,author=?,category=?,total_copies=? WHERE book_id=?"
        )
    ) {

        s.setString(1, title.getText());
        s.setString(2, author.getText());
        s.setString(3, category.getText());

        s.setInt(
            4,
            Integer.parseInt(copies.getText())
        );

        s.setInt(
            5,
            Integer.parseInt(id.getText())
        );

        int n = s.executeUpdate();

        out.setText(
            n > 0
            ? "SUCCESS\nBook updated."
            : "Book not found."
        );

    } catch (Exception ex) {

        out.setText(
            "ERROR: " + ex.getMessage()
        );
    }
});

```

// DELETE BOOK

```
del.addActionListener(e -> {
```

```

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(

```

```

        "DELETE FROM books WHERE book_id=?"
    )
) {

    s.setInt(
        1,
        Integer.parseInt(id.getText())
    );

    int n = s.executeUpdate();

    out.setText(
        n > 0
        ? "SUCCESS\nBook deleted."
        : "Book not found."
    );

} catch (Exception ex) {

    out.setText(
        "ERROR: " + ex.getMessage()
    );
}
});

// SEARCH BOOK

find.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "SELECT * FROM books WHERE book_id=?"
            )
    ) {

        s.setInt(
            1,
            Integer.parseInt(id.getText())
        );

        ResultSet r = s.executeQuery();

        if (r.next()) {

```

```

        title.setText(
            r.getString("title")
        );

        author.setText(
            r.getString("author")
        );

        category.setText(
            r.getString("category")
        );

        copies.setText(
            r.getString("total_copies")
        );

        out.setText(
            "Book found.\nAvailable copies: " +
            r.getInt("available_copies")
        );

    } else {

        out.setText("Book not found.");
    }

} catch (Exception ex) {

    out.setText(
        "ERROR: " + ex.getMessage()
    );
}
});

refresh();
}

// ===== SEARCH BOOK =====

void searchBooks() {

    page(
        "Search Books",
        "Search books by title"
    );
}

```

```

JPanel p = new JPanel(new BorderLayout());

JPanel top = new JPanel();

JTextField search = new JTextField(25);

JButton b = button("Search");

top.add(new JLabel("Title: "));
top.add(search);
top.add(b);

JTextArea out = new JTextArea();

out.setEditable(false);

p.add(top, BorderLayout.NORTH);
p.add(new JScrollPane(out), BorderLayout.CENTER);

main.add(p, BorderLayout.CENTER);

b.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "SELECT * FROM books WHERE title LIKE ?"
            )
    ) {

        s.setString(
            1,
            "%" + search.getText() + "%"
        );

        ResultSet r = s.executeQuery();

        out.setText(
            "ID | TITLE | AUTHOR | CATEGORY | AVAILABLE\n\n"
        );

        while (r.next()) {

            out.append(
                r.getInt("book_id") +

```

```

        " | " +
        r.getString("title") +
        " | " +
        r.getString("author") +
        " | " +
        r.getString("category") +
        " | " +
        r.getInt("available_copies") +
        "\n"
    );
}

} catch (Exception ex) {

    out.setText(
        "ERROR: " + ex.getMessage()
    );
}
});

refresh();
}

// ===== ISSUE BOOK =====

void issueBook() {

    page(
        "Issue Book",
        "Issue an available book to a registered member"
    );

    JPanel p = form();

    JTextField member = new JTextField();
    JTextField book = new JTextField();

    GridBagConstraints g =
        new GridBagConstraints();

    g.insets = new Insets(8, 8, 8, 8);

    row(p, g, 0, "Member ID", member);
    row(p, g, 1, "Book ID", book);

    g.gridx = 0;

```

```

g.gridy = 2;

p.add(
    new JLabel("Due Date"),
    g
);

g.gridx = 1;

p.add(
    new JLabel(
        LocalDate.now()
            .plusDays(14)
            .toString()
    ),
    g
);

JButton x = button("Issue Book");

g.gridx = 1;
g.gridy = 3;

p.add(x, g);

JTextArea out = new JTextArea(6, 55);

g.gridy = 4;

p.add(
    new JScrollPane(out),
    g
);

main.add(p);

x.addActionListener(e -> {

    try (Connection c = con()) {

        int mi =
            Integer.parseInt(member.getText());

        int bi =
            Integer.parseInt(book.getText());

```

```

PreparedStatement cm =
    c.prepareStatement(
        "SELECT member_id FROM members WHERE member_id=?"
    );

cm.setInt(1, mi);

if (!cm.executeQuery().next()) {

    throw new Exception(
        "Member not found"
    );
}

PreparedStatement cb =
    c.prepareStatement(
        "SELECT available_copies FROM books WHERE book_id=?"
    );

cb.setInt(1, bi);

ResultSet rb =
    cb.executeQuery();

if (!rb.next()
    || rb.getInt(1) < 1) {

    throw new Exception(
        "Book unavailable"
    );
}

LocalDate now =
    LocalDate.now();

LocalDate due =
    now.plusDays(14);

PreparedStatement s =
    c.prepareStatement(
        "INSERT INTO issues(member_id,book_id,issue_date,due_date) VALUES(?,?,?,?)"
    );

s.setInt(1, mi);
s.setInt(2, bi);
s.setDate(

```

```

        3,
        Date.valueOf(now)
    );

    s.setDate(
        4,
        Date.valueOf(due)
    );

    s.executeUpdate();

    PreparedStatement u =
        c.prepareStatement(
            "UPDATE books SET available_copies=available_copies-1 WHERE book_id=?"
        );

    u.setInt(1, bi);
    u.executeUpdate();

    out.setText(
        "SUCCESS\n" +
        "Book issued successfully.\n" +
        "Due date: " + due
    );

} catch (Exception ex) {

    out.setText(
        "EXCEPTION HANDLED\n" +
        ex.getMessage()
    );
}

});

refresh();
}

//===== RETURN BOOK =====

void returnBook() {

    page(
        "Return Book & Fine Calculation",
        "Fine = ₹20 per overdue day"
    );
}

```

```

JPanel p = form();

JTextField member = new JTextField();
JTextField book = new JTextField();

GridBagConstraints g =
    new GridBagConstraints();

g.insets = new Insets(8, 8, 8, 8);

row(p, g, 0, "Member ID", member);
row(p, g, 1, "Book ID", book);

JButton x = button("Return Book");

g.gridx = 1;
g.gridy = 2;

p.add(x, g);

JTextArea out = new JTextArea(7, 55);

g.gridy = 3;

p.add(
    new JScrollPane(out),
    g
);

main.add(p);

x.addActionListener(e -> {

    try (Connection c = con()) {

        PreparedStatement s =
            c.prepareStatement(
                "SELECT issue_id,due_date " +
                "FROM issues " +
                "WHERE member_id=? " +
                "AND book_id=? " +
                "AND return_date IS NULL " +
                "ORDER BY issue_id DESC LIMIT 1"
            );

        s.setInt(

```

```

        1,
        Integer.parseInt(member.getText())
    );

    s.setInt(
        2,
        Integer.parseInt(book.getText())
    );

    ResultSet r =
        s.executeQuery();

    if (!r.next()) {

        throw new Exception(
            "No active issue found"
        );
    }

    int issueId =
        r.getInt(1);

    LocalDate due =
        r.getDate(2).toLocalDate();

    LocalDate now =
        LocalDate.now();

    long days =
        Math.max(
            0,
            ChronoUnit.DAYS.between(
                due,
                now
            )
        );

    double fine =
        days * 20;

    PreparedStatement u =
        c.prepareStatement(
            "UPDATE issues SET return_date=?,fine=? WHERE issue_id=?"
        );

    u.setDate(

```

```

        1,
        Date.valueOf(now)
    );

    u.setDouble(
        2,
        fine
    );

    u.setInt(
        3,
        issueId
    );

    u.executeUpdate();

    PreparedStatement q =
        c.prepareStatement(
            "UPDATE books SET available_copies=available_copies+1 WHERE book_id=?"
        );

    q.setInt(
        1,
        Integer.parseInt(book.getText())
    );

    q.executeUpdate();

    out.setText(
        "SUCCESS\n" +
        "Book returned.\n" +
        "Days overdue: " + days +
        "\nFine: ₹" +
        String.format("%.2f", fine)
    );

} catch (Exception ex) {

    out.setText(
        "EXCEPTION HANDLED\n" +
        ex.getMessage()
    );
}
});

refresh();

```

```
}
```

```
// ===== RESERVATION =====
```

```
void reservation() {
```

```
    page(
        "Reservation / Waitlist",
        "Reserve a book or cancel a reservation"
    );
```

```
    JPanel p = form();
```

```
    JTextField member = new JTextField();
    JTextField book = new JTextField();
    JTextField rid = new JTextField();
```

```
    GridBagConstraints g =
        new GridBagConstraints();
```

```
    g.insets = new Insets(8, 8, 8, 8);
```

```
    row(p, g, 0, "Member ID", member);
    row(p, g, 1, "Book ID", book);
    row(p, g, 2, "Reservation ID", rid);
```

```
    JPanel bs = new JPanel();
```

```
    JButton reserve = button("Reserve");
    JButton cancel = button("Cancel");
```

```
    bs.add(reserve);
    bs.add(cancel);
```

```
    g.gridx = 1;
    g.gridy = 3;
```

```
    p.add(bs, g);
```

```
    JTextArea out = new JTextArea(7, 55);
```

```
    g.gridy = 4;
```

```
    p.add(
        new JScrollPane(out),
        g
```

```

);

main.add(p);

reserve.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "INSERT INTO reservations" +
                "(member_id,book_id,reservation_date,status)" +
                " VALUES(?,?,?,?)"
            )
    ) {

        s.setInt(
            1,
            Integer.parseInt(member.getText())
        );

        s.setInt(
            2,
            Integer.parseInt(book.getText())
        );

        s.setDate(
            3,
            Date.valueOf(LocalDate.now())
        );

        s.setString(
            4,
            "WAITLIST"
        );

        s.executeUpdate();

        out.setText(
            "SUCCESS\n" +
            "Reservation added.\n" +
            "Status: WAITLIST"
        );

    } catch (Exception ex) {

```

```

        out.setText(
            "ERROR: " +
            ex.getMessage()
        );
    }
});

cancel.addActionListener(e -> {

    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "UPDATE reservations " +
                "SET status='CANCELLED' " +
                "WHERE reservation_id=?"
            )
    ) {

        s.setInt(
            1,
            Integer.parseInt(rid.getText())
        );

        int n =
            s.executeUpdate();

        out.setText(
            n > 0
            ? "SUCCESS\nReservation cancelled."
            : "Reservation not found."
        );

    } catch (Exception ex) {

        out.setText(
            "ERROR: " +
            ex.getMessage()
        );
    }
});

refresh();
}

```

// ===== OVERDUE =====

```

void notifications() {

    page(
        "Overdue Notifications",
        "Books whose due date has passed"
    );

    JTextArea out =
        new JTextArea();

    out.setEditable(false);

    try (
        Connection c = con();
        Statement s = c.createStatement();
        ResultSet r =
            s.executeQuery(
                "SELECT i.member_id,m.name," +
                "b.title,i.due_date," +
                "GREATEST(DATEDIFF(CURDATE(),i.due_date),0)*20 AS fine " +
                "FROM issues i " +
                "JOIN members m ON i.member_id=m.member_id " +
                "JOIN books b ON i.book_id=b.book_id " +
                "WHERE i.return_date IS NULL " +
                "AND i.due_date<CURDATE()"
            )
    ) {

        out.append(
            "MEMBER | NAME | BOOK | DUE DATE | FINE\n\n"
        );

        while (r.next()) {

            out.append(
                r.getInt(1) +
                " | " +
                r.getString(2) +
                " | " +
                r.getString(3) +
                " | " +
                r.getDate(4) +
                " | ₹" +
                r.getDouble(5) +
                "\n"
            );
        }
    }
}

```

```

        );
    }

    } catch (Exception ex) {

        out.setText(
            "ERROR: " +
            ex.getMessage()
        );
    }

    main.add(
        new JScrollPane(out),
        BorderLayout.CENTER
    );

    refresh();
}

// ===== REPORTS =====

void reports() {

    page(
        "Reports",
        "Inventory, circulation and fine summary"
    );

    JPanel p =
        new JPanel(
            new GridLayout(2, 3, 15, 15)
        );

    p.setBorder(
        BorderFactory.createEmptyBorder(
            20, 25, 25, 25
        )
    );

    String[] labels = {
        "Inventory - Total Copies",
        "Members",
        "Currently Issued",
        "Waiting List",
        "Overdue Books",
        "Total Fine (₹)"
    }

```

```
};
```

```
String[] sql = {  
    "SELECT COALESCE(SUM(total_copies),0) FROM books",  
    "SELECT COUNT(*) FROM members",  
    "SELECT COUNT(*) FROM issues WHERE return_date IS NULL",  
    "SELECT COUNT(*) FROM reservations WHERE status='WAITLIST'",  
    "SELECT COUNT(*) FROM issues WHERE return_date IS NULL AND due_date<CURDATE()",  
    "SELECT COALESCE(SUM(fine),0) FROM issues"  
};
```

```
for (int i = 0; i < labels.length; i++) {
```

```
    JPanel card =  
        new JPanel(  
            new GridLayout(2, 1)  
        );
```

```
    card.setBorder(  
        BorderFactory.createLineBorder(  
            Color.LIGHT_GRAY  
        )  
    );
```

```
    card.add(  
        new JLabel(  
            labels[i],  
            SwingConstants.CENTER  
        )  
    );
```

```
    JLabel value =  
        new JLabel(  
            "0",  
            SwingConstants.CENTER  
        );
```

```
    value.setFont(  
        new Font(  
            "SansSerif",  
            Font.BOLD,  
            24  
        )  
    );
```

```
    try (
```

```

        Connection c = con();
        Statement s = c.createStatement();
        ResultSet r =
            s.executeQuery(sql[i])
    ) {

        r.next();

        value.setText(
            String.valueOf(
                r.getObject(1)
            )
        );

    } catch (Exception ex) {

        value.setText("ERR");
    }

    card.add(value);

    p.add(card);
}

main.add(
    p,
    BorderLayout.CENTER
);

refresh();
}

// ===== JDBC OUTPUT =====

void jdbcOutput() {

    page(
        "JDBC Database Output",
        "Live database information obtained through JDBC"
    );

    JTextArea out =
        new JTextArea();

    out.setEditable(false);

```

```

try (
    Connection c = con();
    Statement s = c.createStatement()
) {

    out.append(
        "JDBC CONNECTION SUCCESSFUL\n"
    );

    out.append(
        "Database: " + DB + "\n"
    );

    out.append(
        "Driver: " +
        c.getMetaData().getDriverName() +
        "\n"
    );

    out.append(
        "Driver Version: " +
        c.getMetaData().getDriverVersion() +
        "\n\n"
    );

    for (
        String t :
        new String[]{
            "members",
            "books",
            "issues",
            "reservations"
        }
    ) {

        ResultSet r =
            s.executeQuery(
                "SELECT COUNT(*) FROM " + t
            );

        r.next();

        out.append(
            "TABLE " +
            t +
            " -> Rows: " +

```

```

        r.getInt(1) +
        "\n"
    );
}

} catch (Exception ex) {

    out.setText(
        "JDBC ERROR\n" +
        ex.getMessage() +
        "\n\nCheck MySQL Server and Connector/J JAR."
    );
}

main.add(
    new JScrollPane(out),
    BorderLayout.CENTER
);

refresh();
}

// ===== EXCEPTION =====

void exceptionTest() {

    page(
        "Exception Handling Test",
        "Demonstrate safe handling of invalid operations"
    );

    JPanel p =
        new JPanel(
            new BorderLayout(10, 10)
        );

    JPanel top = new JPanel();

    JButton invalidMember =
        button("Invalid Member");

    JButton invalidBook =
        button("Invalid Book");

    JButton duplicate =
        button("Duplicate Member");

```

```
top.add(invalidMember);
top.add(invalidBook);
top.add(duplicate);
```

```
p.add(
    top,
    BorderLayout.NORTH
);
```

```
JTextArea out =
    new JTextArea();
```

```
p.add(
    new JScrollPane(out),
    BorderLayout.CENTER
);
```

```
main.add(p);
```

```
invalidMember.addActionListener(e -> {
```

```
    try (
        Connection c = con();
        PreparedStatement s =
            c.prepareStatement(
                "SELECT * FROM members WHERE member_id=?"
            )
    ) {
```

```
        s.setInt(1, -999);
```

```
        if (
            !s.executeQuery().next()
        ) {
```

```
            throw new Exception(
                "Member ID -999 does not exist"
            );
        }
```

```
    } catch (Exception ex) {
```

```
        out.setText(
            "EXCEPTION HANDLED\n" +
            ex.getMessage()
        );
    }
```

```
    );  
    }  
});
```

```
invalidBook.addActionListener(e -> {
```

```
    try (  
        Connection c = con();  
        PreparedStatement s =  
            c.prepareStatement(  
                "SELECT * FROM books WHERE book_id=?"  
            )  
    ) {
```

```
        s.setInt(1, 999999);
```

```
        if (  
            !s.executeQuery().next()  
        ) {
```

```
            throw new Exception(  
                "Book ID 999999 does not exist"  
            );  
        }
```

```
    } catch (Exception ex) {
```

```
        out.setText(  
            "EXCEPTION HANDLED\n" +  
            ex.getMessage()  
        );  
    }
```

```
});
```

```
duplicate.addActionListener(e -> {
```

```
    try (  
        Connection c = con();  
        PreparedStatement s =  
            c.prepareStatement(  
                "INSERT INTO members VALUES" +  
                "(101,'Duplicate'," +  
                "'duplicate@test.com','Student')"  
            )  
    ) {
```

```

        s.executeUpdate();

        out.setText(
            "Inserted member 101.\n" +
            "Run again to demonstrate duplicate-key exception."
        );

    } catch (Exception ex) {

        out.setText(
            "EXCEPTION HANDLED\n" +
            "Duplicate primary key prevented.\n" +
            ex.getMessage()
        );
    }
});

refresh();
}

// ===== MULTITHREADING =====

void multithreading() {

    page(
        "Multithreading Demo",
        "Three background tasks run concurrently"
    );

    JPanel p =
        new JPanel(
            new BorderLayout(10, 10)
        );

    JTextArea out =
        new JTextArea();

    JButton start =
        button("Start Threads");

    p.add(
        new JScrollPane(out),
        BorderLayout.CENTER
    );

    p.add(

```

```

        start,
        BorderLayout.SOUTH
    );

    main.add(p);

    start.addActionListener(e -> {

        out.append(
            "Starting 3 threads...\n"
        );

        for (int i = 1; i <= 3; i++) {

            final int k = i;

            pool.submit(() -> {

                try {

                    Thread.sleep(
                        500 + k * 200
                    );

                    SwingUtilities.invokeLater(
                        () ->
                        out.append(
                            "Thread-" +
                            k +
                            " completed task.\n"
                        )
                    );

                } catch (
                    InterruptedException ignored
                ) {
                }
            });
        }
    });

    refresh();
}

// ===== MAIN =====

```

```

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {

        try {

            initDB();

            new SmartLibrary()
                .setVisible(true);

        } catch (Exception e) {

            JOptionPane.showMessageDialog(
                null,
                "Database Error:\n" +
                e.getMessage() +
                "\n\n" +
                "1. Start MySQL\n" +
                "2. Check USER/PASS\n" +
                "3. Keep Connector/J JAR inside lib folder"
            );
        }
    });
}
}

```